

13.4 Conclusions and Outlook

Now that we've compared the pros and cons in detail, we present our conclusions and our expectations for the coming years in the final section of this book.

13.4.1 Better Results with More Prior Knowledge and Experience

Reporting on AI tools sounds something like this: "Two or three prompts, and the new web application is all set!" This completely overlooks the fact that the efficient use of AI tools only works with prior knowledge. If you've already created a dozen web applications, have fallen into a couple of traps, and are familiar with the usual libraries and tools, then you can easily follow the statements of ChatGPT and others. You can separate the wheat from the chaff in AI-generated code at a glance. However, if you program your first web application, AI tools can at best point you in the right direction. The road remains rocky. The end product won't match the quality of a program that experienced developers would create.

Because the answers from AI tools are always elegantly and politely formulated, it's difficult to distinguish between correct and incorrect answers. It's also difficult to predict in advance which questions the AI tool will excel at and where it will fail.

In other words, AI tools are no substitute for IT training or real-life experience. The more you know or can do yourself, the more AI tools will help you.

The More Incompetent, the More Self-Confident

In psychological literature, the Dunning-Kruger effect (https://en.wikipedia.org/wiki/Dunning%E2%80%93Kruger_effect) describes the overestima-

tion of one's own competence by incompetent people; that is, newcomers often greatly overestimate their own knowledge.

This effect applies exactly to AI! If you assign tasks to AI where you're not competent yourself, you can't assess whether the solutions are reasonable.

13.4.2 Cloud or Local AI?

When we looked more closely at the local execution of language models, we were thrilled. In our view, there are many arguments in favor of executing AI functions for coding locally: data protection, independence from commercial AI providers, costs, and so on.

The “local” execution of the language model doesn't necessarily have to take place on your own computer. A good alternative is a dedicated server set up within the company or in a data center for AI tasks.

However, there are several reasons that speak against the “local is better” or at least “local is safer” approach:

- **Quality**

In our tests, commercial language models such as GPT-4o were clearly superior to the free models. This is not only because these models as such are better. For local execution, you're often forced to use small variants of a free language model due to limited storage space. A local 7b or 15b model can't compete with a model with several hundred billion parameters.

- **Money**

AI companies understandably want to earn money. The easiest way to do this is with cloud subscriptions. For this reason, companies such as OpenAI have no interest in you switching to open-source software and models. They will do anything to ensure that their own commercial AI offering is better than the free competition. And because AI research and training costs a lot of money, it's conceivable that the gap between commercial and free AI tools will widen rather than narrow.

- **Lack of local hardware**

Most computers and notebooks currently on sale (as of early 2025) have

far too little power to run language models locally with sufficient quality and speed.

That will probably change. We expect AI-compatible hardware to become more established. This hardware is characterized by more RAM, higher GPU performance, and the integration of dedicated *neural processing units* (NPUs). AI research is also intensively searching for ways to execute language models more efficiently (new algorithms, more space-saving models, etc.).

Nevertheless, your own laptop or a local AI server will still not be able to keep up with the hardware infrastructure of AI cloud providers in the future.

Based on these points, it can't be ruled out that the cloud approach will prevail—at least for the next few years. An astonishing number of companies already see no problem in storing all of their data in the cloud (Microsoft 365 and others). It's quite possible that this trust will also extend to AI cloud services in the future.

Risk of a Two-Tier IT Society

The majority of our team of authors is at home in the open world. We appreciate that the Linux distribution of our choice, programming languages, developer tools (Docker, Git, etc.), editors, and IDEs are freely available.

Are AI tools throwing a spanner in the works here? As long as free language models aren't competitive or as long as their local execution requires hardware resources that aren't available to private users, there is almost no way around paid AI subscriptions.

You then have the choice: local execution of free language models versus commercial AI cloud offerings. If you can afford it, or if you have no problem with cloud tools seeing all your code, you'll get better AI support than with free, local tools without privacy concerns. We think this is a worrying development.

13.4.3 Quality Enhancement through Training with AI-Generated Code

The first generation of language models was trained exclusively with texts and code written by humans. The more developers use AI tools, the more code repositories, blogs, and online manuals contain code written partly or entirely by AI tools.

Future language models are therefore trained with code produced by a preceding version. To put it even more pointedly: AI trains itself. It's not yet clear how this will affect future language models.

13.4.4 The Coming Few Years

Despite the training problem just mentioned, we expect that future language models will work better than the versions presented in this book. New neural algorithms, new training methods, and new hardware will help.

The question is rather whether AI support for coding will improve in small incremental steps or whether there will be further breakthroughs as in previous years. Of course, we can't say for sure, but we're expecting a somewhat calmer development. From the perspective of this book, our wish list looks something like this:

- **LLMs optimized for coding**

Current language models are *also* able to program. This ability is more of a side effect that results from the way language models work. What we want to see, however, is language models that are designed specifically for coding and that are better suited to the automation of coding tasks. At this point, we've repeatedly come up against the limits of current language models in this book (see in particular [Chapter 10](#) and [Chapter 11](#)).

- **More context**

From an IT perspective, the largest possible context window is desirable so that the language model can ideally take into account all the existing code in a project.

In fact, the size of the context window has recently increased in many LLMs, but at the expense of performance. The more context, the slower the response generation. In our experience, the quality of AI responses also decreases in current models with too much context. To put it casually, too much, too broad a context confuses the model; the answers are less targeted and sometimes have little to do with the question. There is still a lot of room for improvement here.

- **Local LLMs**

Despite all the current limitations, we see a great future in the local execution of language models. Perhaps it will be possible to reduce the size of language models (i.e., to focus on the content that is essential for coding). This would make it possible to achieve a higher execution speed without any loss of quality. The effort to make LLMs smartphone-compatible could help here. If in the future, a smartphone is sufficient to run a local language model satisfactorily, then the same should all the more be the case for a powerful developer notebook.

We're aware that current AI tools weren't primarily created for coding. The focus of further development is also on where the most money can be made, and that is certainly not the comparatively small software developer community. In this respect, we should be a little skeptical as to whether our wishes will come true.

13.4.5 Conclusion

This brings us to the end of this book. Finally, here are four theses:

- **AI tools have become an integral part of coding**

For many developers, the use of AI tools for coding is already just as much a matter of course as the use of the spell-check function in Word or other word processors. This isn't perfect, but useful.

- **Use only within the scope of your own knowledge**

AI tools increase coding efficiency, but can't replace your own basic knowledge. We consider the use of AI-generated code that you don't understand to be highly negligent.

- **Errors keep occurring**

We expect the error rate to decrease somewhat over time, but we're sure that AI tools will continue to produce errors in the future. This is

precisely why it's so important that you understand AI-generated code and take the trouble to test it thoroughly.

- **Seize opportunities and don't dread new developments**

In Europe, we look at the risks of new technologies first and look for regulatory mechanisms. In the United States, it tends to be the other way around, with enthusiasm prevailing. It's certainly debatable which path makes more sense, but it would definitely be a shame to overlook the opportunities that are currently opening up in the AI sector.